



南開大學
Nankai University

计算机学院
并行程序设计实验报告

基于 NTT 的多项式乘法 SIMD 优化

姓名：强徽东

学号：2411764

专业：计算机科学与技术

2026 年 5 月 10 日

目录

1 实验背景	2
2 FFT 介绍	2
3 NTT 原理	3
4 普通 NTT 实现	5
4.1 按位翻转重排	5
4.2 逐层蝶形合并	6
5 Montgomery 规约	8
5.1 引入原因	8
5.2 Montgomery 空间	8
5.3 将取模运算转化为 Montgomery 运算	10
6 NEON SIMD 优化	11
6.1 四路 Montgomery 模乘	11
6.2 向量化蝶形变换	12
6.3 点值相乘与逆变换归一化	13
7 DIT/DIF 优化	14
7.1 基本思想	14
7.2 蝶形公式	14
8 大模数处理	14
8.1 问题分析	14
8.2 其他解决方案	15
9 测试方法	15
10 测试结果与分析	15
11 结论	17

1 实验背景

给定两个长度为 n 的多项式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i,$$

需要计算

$$C(x) = A(x)B(x) = \sum_{k=0}^{2n-2} c_k x^k,$$

其中

$$c_k = \sum_{i+j=k} a_i b_j \pmod{p}.$$

初始代码中的朴素乘法如下：

```

1 void poly_multiply(int *a, int *b, int *ab, int n, int p){
2     for(int i = 0; i < n; ++i){
3         for(int j = 0; j < n; ++j){
4             ab[i+j] = (1LL * a[i] * b[j] % p + ab[i+j]) % p;
5         }
6     }
7 }
```

该算法执行 n^2 次乘加。当 $n = 131072$ 时，计算量约为 1.7×10^{10} ，因此需要使用快速算法和硬件并行优化。

2 FFT 介绍

FFT (Fast Fourier Transform, 快速傅里叶变换) 是一种快速计算离散傅里叶变换的算法，常用于信号处理、卷积计算和多项式乘法等场景。在多项式乘法中，FFT 的核心思想是将多项式从系数表示转换为点值表示。

设有两个多项式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i.$$

若直接按照定义计算

$$C(x) = A(x)B(x),$$

则需要计算卷积

$$c_k = \sum_{i+j=k} a_i b_j,$$

时间复杂度为 $O(n^2)$ 。

FFT 利用点值表示降低计算复杂度。一个次数小于 n 的多项式可以由 n 个不同点上的取值唯一确定。因此，多项式乘法可以转化为：

系数表示 $\rightarrow$ 点值表示 $\rightarrow$ 逐点相乘 $\rightarrow$ 系数表示。

在点值表示下，只需要对每个点分别相乘：

$$C(x_i) = A(x_i)B(x_i).$$

FFT 选择复数单位根作为求值点。长度为 N 的单位根为

$$\omega_N = e^{2\pi i/N},$$

它满足

$$\omega_N^N = 1.$$

利用单位根的对称性和周期性，FFT 可以通过分治将多点求值和插值的复杂度降为

$$O(N \log N).$$

不过，FFT 使用复数和浮点数计算，实际实现中可能产生精度误差。当需要在模意义下进行精确的多项式乘法时，通常使用 NTT 来替代 FFT。NTT 可以看作是模整数域上的 FFT，它使用模意义下的单位根，避免了浮点误差。

关于 FFT 的更多原理介绍可参考：[FFT 原理介绍](#)。

3 NTT 原理

NTT (Number Theoretic Transform, 数论变换) 可以看作是在模意义下的 FFT。FFT 使用复数单位根，而 NTT 使用模 p 意义下的单位根。由于 NTT 全程在整数模域中运算，因此不会产生浮点误差，特别适合本实验中的模意义多项式乘法。

设有两个多项式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i.$$

朴素乘法需要计算

$$c_k = \sum_{i+j=k} a_i b_j \pmod{p},$$

时间复杂度为 $O(n^2)$ 。NTT 同样采用点值表示思想：

系数表示 $\rightarrow$ 点值表示 $\rightarrow$ 逐点相乘 $\rightarrow$ 系数表示。

在点值表示下，有

$$C(\omega_i) = A(\omega_i)B(\omega_i) \pmod{p}.$$

与 FFT 不同，NTT 中的单位根不是复数，而是模 p 意义下的数。长度为 N 的单位根 ω_N 需要满足

$$\omega_N^N \equiv 1 \pmod{p},$$

并且对于 $0 < k < N$ ，有

$$\omega_N^k \not\equiv 1 \pmod{p}.$$

这要求

$$N \mid (p-1).$$

因此, NTT 常用模数通常满足

$$p = c \cdot 2^m + 1.$$

这样当变换长度 N 是不超过 2^m 的二次幂时, 就可以找到对应的单位根。

若 g 是模 p 的原根, 则长度为 N 的单位根可以由

$$\omega_N = g^{(p-1)/N} \pmod{p}$$

得到。实验数据保证给定模数的原根均为 3, 因此在长度为 N 的 NTT 中可以取

$$\omega_N = 3^{(p-1)/N} \pmod{p}.$$

NTT 的分治过程与 FFT 完全类似。将多项式按下标奇偶拆分:

$$A(x) = A_0(x^2) + xA_1(x^2).$$

其中

$$A_0(x) = a_0 + a_2x + a_4x^2 + \cdots,$$

$$A_1(x) = a_1 + a_3x + a_5x^2 + \cdots.$$

对 $x = \omega_N^k$, 有

$$A(\omega_N^k) = A_0(\omega_N^{2k}) + \omega_N^k A_1(\omega_N^{2k}).$$

又因为模意义下的单位根同样满足

$$\omega_N^{k+N/2} \equiv -\omega_N^k \pmod{p},$$

所以

$$A(\omega_N^{k+N/2}) = A_0(\omega_N^{2k}) - \omega_N^k A_1(\omega_N^{2k}) \pmod{p}.$$

因此 NTT 的蝶形操作为:

$$u = f[i+j],$$

$$v = w \cdot f[i+j+N/2] \pmod{p},$$

$$f[i+j] = u + v \pmod{p},$$

$$f[i+j+N/2] = u - v \pmod{p}.$$

在程序实现中, 为了保证结果始终落在 $[0, p)$ 内, 需要对加减结果进行修正:

$$u + v \geq p \Rightarrow u + v - p,$$

$$u - v < 0 \Rightarrow u - v + p.$$

这样可以避免负数或超过模数范围的结果继续参与后续计算。

逆 NTT 与正 NTT 类似，只是使用单位根的逆元：

$$\omega_N^{-1}.$$

由于模数 p 为质数，可以根据费马小定理求逆元：

$$x^{-1} \equiv x^{p-2} \pmod{p}.$$

逆变换结束后，还需要乘上 N^{-1} ：

$$f_i = f_i \cdot N^{-1} \pmod{p}.$$

完整的 NTT 多项式乘法流程如下。首先选择最小的二次幂 N ，满足

$$N \geq 2n - 1.$$

然后将两个多项式补零到长度 N ，分别进行正 NTT：

$$\hat{A} = \text{NTT}(A),$$

$$\hat{B} = \text{NTT}(B).$$

接着逐点相乘：

$$\hat{C}_i = \hat{A}_i \hat{B}_i \pmod{p}.$$

最后进行逆 NTT：

$$C = \text{INTT}(\hat{C}).$$

最终得到的前 $2n - 1$ 项即为多项式乘法结果。

本实验中的前三个模数

$$7340033, \quad 104857601, \quad 469762049$$

均在 32 位整数范围内，可以使用 32 位 NTT 和 Montgomery32 进行优化。第四个模数

$$263882790666241 = 15 \cdot 2^{44} + 1$$

同样满足 NTT 对模数形式的要求，因此也可以进行 NTT；但它超过 32 位整数范围，并且乘法中间结果会超过 64 位，因此需要使用 64 位存储和 `__uint128_t` 或 Montgomery64 进行模乘。

4 普通 NTT 实现

普通迭代 NTT 通常由两个主要步骤组成：首先对数组进行按位翻转重排，使数据顺序满足迭代蝶形合并的要求；然后按照长度 $2, 4, 8, \dots, N$ 逐层进行蝶形合并。下面分别介绍这两个部分。

4.1 按位翻转重排

在递归形式的 NTT 中，多项式会不断按照下标奇偶拆分。例如长度为 8 的数组，拆分顺序本质上对应下标二进制的反转。为了用迭代方式模拟递归过程，需要在正式进行蝶形合并之前，先将数组

重排到按位翻转顺序。

设变换长度为 $N = 2^m$ ，下标 i 的二进制表示为

$$i = (b_{m-1}b_{m-2} \cdots b_1b_0)_2.$$

其按位翻转后的下标为

$$\text{rev}(i) = (b_0b_1 \cdots b_{m-2}b_{m-1})_2.$$

例如当 $N = 8$ 时，下标需要使用 3 位二进制表示：

$$0 = (000)_2 \rightarrow (000)_2 = 0,$$

$$1 = (001)_2 \rightarrow (100)_2 = 4,$$

$$2 = (010)_2 \rightarrow (010)_2 = 2,$$

$$3 = (011)_2 \rightarrow (110)_2 = 6,$$

$$4 = (100)_2 \rightarrow (001)_2 = 1,$$

$$5 = (101)_2 \rightarrow (101)_2 = 5,$$

$$6 = (110)_2 \rightarrow (011)_2 = 3,$$

$$7 = (111)_2 \rightarrow (111)_2 = 7.$$

因此重排顺序为

$$0, 4, 2, 6, 1, 5, 3, 7.$$

代码中的按位翻转重排如下：

```

1  for (int i = 1, j = 0; i < n; ++i) {
2      int bit = n >> 1;
3      while (j & bit) {
4          j ^= bit;
5          bit >>= 1;
6      }
7      j ^= bit;
8      if (i < j) std::swap(f[i], f[j]);
9  }
```

其中 j 表示当前下标 i 的按位翻转结果。若 $i < j$ ，则交换 $f[i]$ 和 $f[j]$ ，从而避免重复交换。完成按位翻转后，数组中相邻元素就对应递归分治中的最小子问题，可以直接进行迭代式蝶形合并。

4.2 逐层蝶形合并

完成按位翻转重排后，NTT 从长度为 2 的小区间开始合并，然后依次合并长度为 4, 8, ..., N 的区间。代码结构如下：

```

1  for (int len = 2; len <= n; len <<= 1) {
2      int half = len >> 1;
```

```

3   int wn = qpow(3, (p - 1) / len, p);
4
5   for (int i = 0; i < n; i += len) {
6       int w = 1;
7       for (int j = 0; j < half; ++j) {
8           ...
9           w = 1LL * w * wn % p;
10      }
11  }
12 }

```

其中 `len` 表示当前正在合并的区间长度, `half = len / 2` 表示左右两半区间的长度。每一层使用的单位根为

$$wn = 3^{(p-1)/len} \pmod{p}.$$

因为实验中的模数均以 3 为原根, 所以可以通过快速幂得到当前层的主单位根。

对于长度为 `len` 的阶段, 蝶形操作为:

$$\begin{aligned} u &= f[i + j], \\ v &= w \cdot f[i + j + \text{half}] \pmod{p}, \\ f[i + j] &= u + v \pmod{p}, \\ f[i + j + \text{half}] &= u - v \pmod{p}. \end{aligned}$$

其中 w 是当前使用的旋转因子。随着 j 增加, w 依次取:

$$1, wn, wn^2, \dots, wn^{\text{half}-1}.$$

为了保证结果始终位于 $[0, p)$, 程序中不会直接使用普通加减, 而是进行范围修正:

```

1   int x = u + v;
2   if (x >= p) x -= p;
3
4   int y = u - v;
5   if (y < 0) y += p;

```

这样可以避免出现超过模数或负数的中间结果。

蝶形合并的数学来源是单位根的对称性。对于长度为 len 的变换, 左右两半可以看作两个长度为 $len/2$ 的子变换结果。设

$$u = A_0(\omega^{2j}), \quad f[i + j + \text{half}] = A_1(\omega^{2j}),$$

并令

$$w = \omega^j,$$

则

$$v = wA_1(\omega^{2j}).$$

由单位根性质可得：

$$A(\omega^j) = u + v,$$

$$A(\omega^{j+\text{half}}) = u - v.$$

因此，一次蝶形操作可以同时得到两个点值结果。

逐层蝶形合并每一层都处理 N 个元素，共有 $\log N$ 层，因此 NTT 的总复杂度为

$$O(N \log N).$$

逆 NTT 与正 NTT 的结构基本一致，区别在于使用单位根的逆元：

$$wn^{-1} \equiv wn^{p-2} \pmod{p}.$$

此外，逆变换结束后还需要乘以 N^{-1} ：

$$f_i = f_i \cdot N^{-1} \pmod{p}.$$

在模 p 意义下，除以 N 等价于乘以 N 的乘法逆元：

$$N^{-1} \equiv N^{p-2} \pmod{p}.$$

5 Montgomery 规约

5.1 引入原因

NTT 中最频繁的操作是模乘。普通写法

$$a \cdot b \bmod p$$

包含除法意义下的取模，代价较高，也不适合 SIMD。Montgomery 规约通过引入 $R = 2^{32}$ 或 $R = 2^{64}$ ，把取模转换为乘法、加法、移位和条件减法。

5.2 Montgomery 空间

Montgomery 规约的核心思想是：不直接在普通整数表示下进行模乘，而是先将数映射到一个特殊的表示空间中，在该空间内完成乘法，从而避免频繁使用普通取模运算。

设模数为 p ，选择一个整数 R ，满足

$$R > p, \quad \gcd(R, p) = 1.$$

在程序中通常取

$$R = 2^{32}$$

或

$$R = 2^{64},$$

因为此时除以 R 可以通过右移完成。

对于普通整数 x ，定义它在 Montgomery 空间中的表示为

$$\bar{x} \equiv xR \pmod{p}.$$

若两个数 x, y 的 Montgomery 表示分别为

$$\bar{x} \equiv xR \pmod{p}, \quad \bar{y} \equiv yR \pmod{p},$$

则二者直接相乘得到

$$\bar{x}\bar{y} \equiv xyR^2 \pmod{p}.$$

但我们希望乘法结果仍然保持 Montgomery 表示，即得到

$$\overline{xy} \equiv xyR \pmod{p}.$$

因此需要一个规约函数 `reduce`，用于消去一个 R ：

$$\text{reduce}(T) \equiv TR^{-1} \pmod{p}.$$

于是

$$\text{reduce}(\bar{x}\bar{y}) \equiv xyR^2R^{-1} \equiv xyR \pmod{p}.$$

这说明 Montgomery 空间中的乘法可以定义为

$$\bar{x} \otimes \bar{y} = \text{reduce}(\bar{x}\bar{y}),$$

其结果仍然在 Montgomery 空间中，可以继续参与后续运算。

在代码中，普通数 x 需要先通过 `init(x)` 转入 Montgomery 空间：

$$x \xrightarrow{\text{init}} xR \bmod p.$$

计算结束后，再通过 `value(x)` 转回普通表示：

$$xR \xrightarrow{\text{value}} x \bmod p.$$

在 NTT 中，蝶形变换包含大量模乘。将数组和旋转因子都放入 Montgomery 空间后，模乘可以用 `reduce` 高效完成；同时加法和减法仍然成立：

$$\bar{x} + \bar{y} \equiv (x + y)R \pmod{p},$$

$$\bar{x} - \bar{y} \equiv (x - y)R \pmod{p}.$$

因此，NTT 可以整体在 Montgomery 空间中进行计算。

5.3 将取模运算转化为 Montgomery 运算

进一步地, Montgomery 规约的关键在于如何高效实现

$$\text{reduce}(T) \equiv TR^{-1} \pmod{p}.$$

直接计算 $TR^{-1} \bmod p$ 仍然需要取模, 因此 Montgomery 方法通过构造一个辅助数 q , 使得

$$T + qp$$

能够被 R 整除。这样就可以写成

$$\text{reduce}(T) = \frac{T + qp}{R}.$$

由于

$$T + qp \equiv T \pmod{p},$$

所以

$$\frac{T + qp}{R} \equiv TR^{-1} \pmod{p}.$$

为了使 $T + qp$ 能够被 R 整除, 需要满足

$$T + qp \equiv 0 \pmod{R}.$$

即

$$qp \equiv -T \pmod{R}.$$

因为 $\gcd(p, R) = 1$, 所以 p 在模 R 意义下存在逆元。令

$$p' \equiv -p^{-1} \pmod{R},$$

则可以取

$$q \equiv Tp' \pmod{R}.$$

当 $R = 2^{32}$ 或 $R = 2^{64}$ 时, 对 R 取模只需要保留低 32 位或低 64 位, 因此计算 q 可以通过整数自然溢出或截断完成。

以 32 位版本为例, 代码中的规约函数为:

```

1 u32 reduce(u64 x) const {
2     u32 q = (u32)((x & 0xffffffffULL) * inv);
3     u64 y = (x + u64(q) * mod) >> 32;
4     if (y >= mod) y -= mod;
5     return u32(y);
6 }

```

其中 `inv` 对应

$$p' \equiv -p^{-1} \pmod{2^{32}}.$$

表达式

$$x \& 0xffffffff$$

用于取得 x 的低 32 位，即

$$T \bmod R.$$

然后计算

$$q \equiv (T \bmod R)p' \pmod{R}.$$

接着

$$x + qp$$

被构造为 R 的倍数，因此可以通过右移完成除以 R ：

$$y = \frac{x + qp}{2^{32}}.$$

最后，如果 $y \geq p$ ，再减去一次 p ，使结果落在 $[0, p)$ 范围内：

$$y \leftarrow y - p.$$

因此，普通模乘中的取模操作被转化为了

乘法 + 加法 + 位移 + 条件减法。

这正是 Montgomery 规约适合 SIMD 优化的原因。

6 NEON SIMD 优化

在完成 Montgomery 规约后，本实验进一步使用 ARM NEON 对 NTT 中的关键模乘和蝶形操作进行向量化。由于前三个模数均在 32 位范围内，数据可以使用 `uint32_t` 存储，因此 NEON 的 128 位寄存器可以一次处理 4 个 32 位元素。

NTT 中最核心的计算出现在蝶形变换中：

$$v_i = w_i \cdot right_i \pmod{p},$$

$$left'_i = left_i + v_i \pmod{p},$$

$$right'_i = left_i - v_i \pmod{p}.$$

因此，优化的重点是一次并行处理 4 个蝶形中的模乘、模加和模减。

6.1 四路 Montgomery 模乘

NEON 版 Montgomery 模乘的核心函数为：

```
1 static inline uint32x4_t montgomery_mul4_neon(
2     uint32x4_t a, uint32x4_t b, const Montgomery32& mont)
```

该函数输入两个 4 路 32 位向量：

$$a = [a_0, a_1, a_2, a_3],$$

$$b = [b_0, b_1, b_2, b_3],$$

目标是并行计算：

$$a_i b_i \pmod{p}, \quad i = 0, 1, 2, 3.$$

由于 `uint32_t * uint32_t` 的中间结果是 64 位且 reduce 过程中必须要使用到 64 位值的高 32 位，而 NEON 的 128 位寄存器只能容纳两个 64 位元素，因此 4 路乘法需要拆成低两路和高两路分别处理：

```

1 uint64x2_t prod_low = vmull_u32(
2     vget_low_u32(a),
3     vget_low_u32(b)
4 );
5
6 uint64x2_t prod_high = vmull_u32(
7     vget_high_u32(a),
8     vget_high_u32(b)
9 );

```

其中 `prod_low` 保存

$$[a_0 b_0, a_1 b_1],$$

`prod_high` 保存

$$[a_2 b_2, a_3 b_3].$$

随后分别对这两个 `uint64x2_t` 进行 Montgomery 规约，将 64 位乘积规约回 32 位结果：

```

1 uint32x2_t low = montgomery_reduce2_neon(prod_low, mont.mod, mont.inv);
2 uint32x2_t high = montgomery_reduce2_neon(prod_high, mont.mod, mont.inv);
3 return vcombine_u32(low, high);

```

这样即可得到 4 路并行的 Montgomery 模乘结果。

6.2 向量化蝶形变换

在蝶形变换中，每 4 个连续位置可以组成一组 SIMD 计算。首先构造 4 个旋转因子：

$$roots = [w, w \cdot wn, w \cdot wn^2, w \cdot wn^3],$$

并加载右半部分的 4 个元素：

$$right = [right_0, right_1, right_2, right_3].$$

然后使用 NEON Montgomery 模乘得到：

$$v = roots \cdot right \pmod{p}.$$

对应代码为：

```

1 uint32x4_t roots = vld1q_u32(roots_buf);

```

```

2 uint32x4_t right = vreinterpretq_u32_s32(
3     vld1q_s32(reinterpret_cast<const int32_t*>(&f[i + j + half]))
4 );
5 uint32x4_t v = montgomery_mul4_neon(roots, right, mont);

```

得到 v 后, 再对左半部分进行 4 路模加和模减:

$$left' = left + v \pmod{p},$$

$$right' = left - v \pmod{p}.$$

模加通过比较结果是否大于等于 p 来决定是否减去 p :

```

1 uint32x4_t sum = vaddq_u32(u, v);
2 uint32x4_t sum_ge_mod = vcgeq_u32(sum, mod_v);
3 sum = vsubq_u32(sum, vandq_u32(sum_ge_mod, mod_v));

```

模减则通过判断 $u < v$ 来决定是否加回 p :

```

1 uint32x4_t diff = vsubq_u32(u, v);
2 uint32x4_t diff_need_mod = vcltq_u32(u, v);
3 diff = vaddq_u32(diff, vandq_u32(diff_need_mod, mod_v));

```

这样, 一次 NEON 操作即可完成 4 个蝶形中的模乘、模加和模减。

6.3 点值相乘与逆变换归一化

除了蝶形变换外, NTT 多项式乘法中的点值相乘阶段也包含大量模乘:

$$A_i = A_i B_i \pmod{p}.$$

因此, 本实验同样使用 `montgomery_mul4_neon` 对该阶段进行 4 路并行处理:

```

1 uint32x4_t va = vld1q_u32(...);
2 uint32x4_t vb = vld1q_u32(...);
3 uint32x4_t vc = montgomery_mul4_neon(va, vb, mont);

```

逆 NTT 结束后需要乘以 n^{-1} , 即

$$f_i = f_i \cdot n^{-1} \pmod{p}.$$

这一阶段也可以将 n^{-1} 复制成 4 路向量:

$$[n^{-1}, n^{-1}, n^{-1}, n^{-1}],$$

然后使用同样的 4 路 Montgomery 模乘完成归一化。

综上, NEON SIMD 优化主要覆盖了 NTT 中最频繁的三类操作: 蝶形中的旋转因子乘法、点值相乘、以及逆变换后的归一化乘法。通过将这些模乘改写为 4 路 Montgomery SIMD 运算, 可以更充分地利用 ARM NEON 的数据并行能力。

7 DIT/DIF 优化

7.1 基本思想

普通迭代 NTT 通常在变换前进行 bit reversal。DIT/DIF 优化利用 DIT 和 DIF 的输入输出顺序互补，减少显式位翻转开销。

本实验前面就是先位反转然后进行 DIT

而 DIF 是 NTT/FFT 的另一种迭代实现方式。与常见的 DIT，DIF 通常从较大的区间开始拆分，逐层向较小区间递推。DIF 的一个重要特点是：自然顺序输入经过 DIF 变换后，输出通常是按位翻转顺序。

所以一种常见组合为：

DIF 正变换：自然序输入，位翻转频域输出，

DIT 逆变换：位翻转频域输入，自然序输出。

于是多项式乘法流程变为：

$$A \rightarrow \text{DIF NTT}, \quad B \rightarrow \text{DIF NTT},$$

$$C_i = A_i B_i,$$

$$C \rightarrow \text{DIT inverse NTT}.$$

由于 A 和 B 的频域结果具有相同的位翻转顺序，因此逐点相乘仍然正确。这样可以避免每次 NTT 开头显式执行随机访存性质的 bit reversal。

7.2 蝶形公式

DIT 蝶形为：

$$v = w \cdot \text{right}, \quad \text{left}' = u + v, \quad \text{right}' = u - v.$$

DIF 蝶形为：

$$\text{left}' = u + v, \quad \text{right}' = (u - v) \cdot w.$$

实验代码中为 DIF 新增了向量化蝶形函数。

8 大模数处理

8.1 问题分析

第四个模数

$$263882790666241$$

超过 `int` 和 32 位无符号整数范围。虽然它可以放入 64 位整数，但乘法中间结果满足

$$p^2 \approx 6.96 \times 10^{28},$$

远超 64 位范围。因此大模数版本必须使用 `__uint128_t` 或 64 位 Montgomery。本实验中采取 64 位 Montgomery 的方案，既能处理大模数又能利用 NEON 进行部分优化。

大模数版本使用 $R = 2^{64}$ 的 Montgomery64 与 Montgomery32 类似，但需要处理 128 位的乘积和 64 位的规约结果。只需要将 32 位版本中的 32 位替换成 64 位，64 位替换成 128 位即可。

对于 64 位数据，NEON 128 位寄存器一次只能容纳 2 个 `uint64_t`。实验实现了两路并行的蝶形加减：

```
1 static inline void butterfly_addsub2_neon_u64(
2     u64 *left, u64 *right, const u64 v_buf[2], u64 mod)
```

需要说明的是，大模数版本中的 64 位 Montgomery 模乘仍然是标量的。这是因为完整的 `uint64_t * uint64_t -> uint128_t` 在 NEON 中没有像 32 位 `vmull_u32` 那样直接方便的向量指令。若要实现真正的两路 64 位 Montgomery 模乘，需要手工拆成 32 位高低位并处理进位，复杂度较高。

8.2 其他解决方案

对于大模数，也可以不直接在该模数下进行 NTT，而是选择多个较小的 NTT 友好模数，分别计算多项式乘法结果。由于这些小模数两两互质，可以使用中国剩余定理将多个小模数下的结果合并，再对目标大模数取模。

这种方法的优点是可以复用 32 位 NTT 和 SIMD Montgomery 优化，避免直接处理大模数下的 64 位乘法和 128 位中间结果。缺点是需要进行多次 NTT，并且最后要额外进行 CRT 合并，代码复杂度和常数开销更高。因此，本实验主要采用直接的大模数 Montgomery64 实现。

9 测试方法

建议在 ARM NEON 环境下使用：

```
1 g++ -O2 main.cc -o main
2 sh test.sh 1 1
```

前三个 32 位模数可使用

```
1 poly_multiply_ntt_ditdif_simd
```

第四个大模数应使用

```
1 poly_multiply_ntt_u64_simd
```

10 测试结果与分析

本实验分别测试了四个版本的 NTT 实现：

1. 普通 NTT；
2. Montgomery 规约取模，并使用 NEON 对蝶形加减并行；
3. Montgomery 规约取模，并进一步使用 NEON 对模乘进行并行；

4. 在版本 3 的基础上使用 DIT/DIF 优化, 减少显式位翻转开销。

测试结果如下表所示。

模数	普通 NTT	加减并行	模乘并行	DIT/DIF 优化
7340033	55.561	47.9239	50.0162	42.2464
104857601	59.6159	48.4696	50.1707	41.933
469762049	62.7503	48.5684	50.3972	42.4692

表 1: 不同版本在 $n = 131072$ 下的运行时间

从结果可以看出, 普通 NTT 的平均运行时间约为

$$59.31 \text{ ms.}$$

在加入 Montgomery 规约并对蝶形加减进行 NEON 并行后, 平均运行时间下降到

$$48.32 \text{ ms,}$$

相比普通 NTT 提升约

$$\frac{59.31}{48.32} \approx 1.23$$

倍。这说明 Montgomery 规约和 SIMD 加减优化能够有效减少蝶形变换中的取模和加减开销。

第三个版本进一步对 Montgomery 模乘进行了 NEON 向量化, 平均运行时间约为

$$50.19 \text{ ms.}$$

该版本相比普通 NTT 仍有明显提升, 但略慢于仅加减并行版本。原因可能是: 虽然模乘被改写为 SIMD 形式, 但 32 位乘法需要扩展到 64 位中间结果, 4 路模乘必须拆成低两路和高两路分别规约, 额外的数据拆分、合并和寄存器操作带来了一定开销。因此, 在当前实现中, 模乘并行并没有完全转化为更低的实际运行时间。此外, 旋转因子四路加载的过程也需要额外的计算, 这也增加降低了运行效率。

第四个版本在版本 3 的基础上加入 DIT/DIF 优化后, 平均运行时间进一步下降到

$$42.22 \text{ ms.}$$

相比普通 NTT, 整体提升约

$$\frac{59.31}{42.22} \approx 1.41$$

倍; 相比版本 3, 提升约

$$\frac{50.19}{42.22} \approx 1.19$$

倍。说明 DIT/DIF 组合减少了显式位翻转带来的额外访存开销, 对整体性能有较明显改善。

小规模样例 $n = 4$ 中, SIMD 版本并不占优势。这是因为小规模数据无法充分利用向量并行能力, 函数调用、初始化、单位根计算等固定开销反而占主要部分。因此 SIMD 优化主要在大规模输入下体现效果。

总体来看, 本实验的优化效果较为明显。Montgomery 规约降低了普通取模的代价, NEON SIMD 提供了数据级并行能力, 而 DIT/DIF 进一步减少了位翻转带来的访存开销。在三个 32 位模数测试中, 最终版本相较普通 NTT 平均获得约 1.4 倍的性能提升。

11 结论

本实验从朴素多项式乘法出发，逐步实现了 NTT、Montgomery 规约、NEON 蝶形向量化、NEON Montgomery 模乘、DIT/DIF 顺序优化以及大模数 NTT。对于 32 位模数，最终版本已经覆盖了 NTT 中主要的模乘、模加、模减热点；对于大模数，受限于 64 位乘法需要 128 位中间结果，本实验采用 Montgomery64 标量模乘与两路 NEON 蝶形加减的折中方案，保证正确性的同时保留了一部分 SIMD 优化空间。

[本实验的 github 仓库](#)

AI 问答记录见提交的另一个文件